

ASTRONOMY, COORDINATE SYSTEMS & OBSERVATION GEOMETRY

A Comprehensive Standalone Beginner Guide to Physical Observation Mechanics, Coordinate Transformations, & Orbital Geometry

Target: Pipeline Contributors **Focus:** Coordinates, Math & Geometry **Format:** Multi-Page Reference Guide

01 WHY DO WE NEED ASTRONOMY FOR TERMINATOR MOON?

Terminator Moon is not simply an image-processing project. Our input is an image of the Moon, but the image is the result of a physical observation geometry. At a particular time:

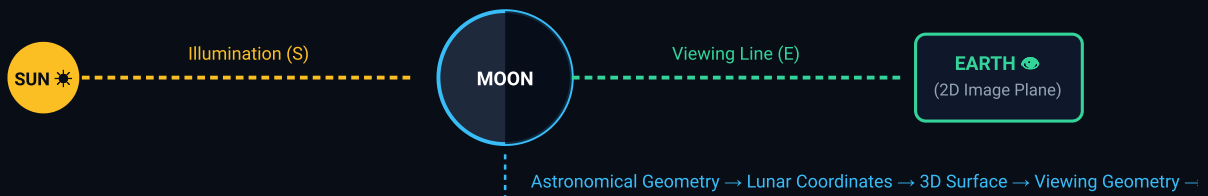
- The Moon has a particular orientation in space;
- Earth is located in a particular direction relative to the Moon;
- The Sun is located in another direction;
- The Moon is illuminated from the Sun's direction;
- Only part of the surface is visible from Earth;
- The visible part is projected onto a 2-D image;
- The boundary between illuminated and dark terrain forms the **terminator**.

Therefore, when comparing two images, we need to understand: *Where is the observer? Where is the Sun? How is the Moon oriented? Which lunar surface point corresponds to each image pixel?* This is the fundamental geometry of the project.

02 THE MOST IMPORTANT MENTAL MODEL

The computer sees only the final 2-D image. But the image was produced by a 3-D astronomical situation. Our job is to understand enough of that geometry to move across the spatial processing chain:

FIGURE 1: PHYSICAL OBSERVATION GEOMETRY CHAIN



03 THREE DIFFERENT "WORLDS" OF COORDINATES

One of the easiest ways for a new contributor to become confused is to mix different coordinate systems. Terminator Moon involves at least three conceptual coordinate spaces:

3.1 Lunar Surface Coordinates

Describes locations directly on the Moon surface using angles: `longitude = 20°`, `latitude = -10°`.

3.2 3-D Cartesian Space

Point represented as $P = (x, y, z)$. Essential for vector geometry involving Sun/Earth directions, normals, and rotations.

3.3 Image Pixel Coordinates

2-D grid locations in pixels: $(x, y) = (1037, 542)$. This is simply a location in the image, not automatically latitude or longitude.

04 THE MOON AS A COORDINATE SYSTEM (SELENOGRAPHIC)

To describe a location on the Moon, astronomers use a lunar coordinate system generally called a **selenographic coordinate system** ("selenographic" means relating to mapping the Moon, analogous to geographic coordinates on Earth).

Lunar Latitude (φ)

Tells us how far north or south a point is relative to the lunar equator.

- `latitude > 0` → Northern hemisphere (+90° = North Pole)
- `latitude = 0` → Lunar equator
- `latitude < 0` → Southern hemisphere (-90° = South Pole)

Lunar Longitude (λ) & Prime Meridian

Tells us where a point lies east or west around the Moon relative to the defined **Lunar Prime Meridian** (`longitude = 0°`).

Rule: Coordinate sign conventions depend on the reference system —never assume sign without checking specs!

LATITUDE + LONGITUDE = A SURFACE LOCATION

Together, (`latitude = +12°`, `longitude = -35°`) identify one exact physical location on the lunar surface, directly analogous to Earth coordinates like (`7.29° N`, `80.63° E`).

05 DEGREES, RADIANs & NON-CARTESIAN GEOMETRY

Degrees Are Angles: A complete circle is 360°, half is 180°, and 1° = $\pi/180$ radians. This is critical in Python code because trigonometric functions require radians:

```
import math
angle_deg = 30
angle_rad = math.radians(angle_deg) # gives angle_rad ≈ 0.5236
```

Latitude/Longitude Are NOT Cartesian Coordinates:

You cannot perform ordinary 2-D geometry such as `x = longitude` and `y = latitude` to represent physical distance. Because the Moon is spherical, longitude lines converge toward the poles:

- Near the equator: 1° longitude corresponds to a relatively large physical distance (~30.3 km).
- Near a pole: 1° longitude corresponds to a much smaller physical distance, converging to zero at the pole.

06 FROM SPHERICAL TO 3-D CARTESIAN COORDINATES

For geometry calculations, we represent a lunar surface point as a 3-D vector. For a spherical Moon of radius R , latitude φ , and longitude λ (in radians):

```
x = R * cos(phi) * cos(lam)
y = R * cos(phi) * sin(lam)
z = R * sin(phi)
P = (x, y, z) # 3-D representation of the surface point
```

Why Cartesian Coordinates Matter to Terminator Moon:

If we know the Sun direction unit vector $\mathbf{S} = (S_x, S_y, S_z)$ and the surface-normal direction unit vector $\mathbf{N} = (N_x, N_y, N_z)$, the angle θ between them is calculated using the dot product:

$$\mathbf{N} \cdot \mathbf{S} = |\mathbf{N}| |\mathbf{S}| \cos \theta = \cos \theta \quad (\text{for unit vectors } |\mathbf{N}| = |\mathbf{S}| = 1)$$

07 SURFACE NORMALS & THE GEOMETRY OF THE TERMINATOR

At every point on the surface of a smooth Moon, the **surface normal (N)** is an imaginary perpendicular arrow pointing outward from the center.

$N \cdot S > 0$

Surface faces toward the Sun. Illuminated daylight region.

$N \cdot S = 0$

Surface perpendicular to sunlight ($\cos 90^\circ = 0$). Geometric **terminator**.

$N \cdot S < 0$

Surface faces away from Sun. Shadowed night region.

WHY THE TERMINATOR APPEARS CURVED IN 2D

The 3-D terminator is a continuous geometric loop around the Moon. When viewed from Earth, the camera captures a 2-D perspective projection of this 3-D ring, turning it into a curved boundary on the screen.

08 SUBSOLAR POINT VS. SUB-EARTH POINT

Two critical landmark points link orbital geometry with metadata coordinates:

The Subsolar Point

The surface location where the Sun is directly overhead (zenith angle = 0°). Expressed via metadata as `subsolar_latitude` and `subsolar_longitude`. Dictates illumination direction.

The Sub-Earth Point

The surface location where Earth is directly overhead. Expressed via metadata as `subearth_latitude` and `subearth_longitude`. Dictates viewing perspective.

Sub-Earth Point $\neq$ Image Pixel Center: The sub-Earth point is a physical surface location, whereas the disk center is an image projection concept. Real viewing geometry, libration, and shape separate them.

09 WHAT IS LIBRATION & WHY IT MATTERS

The Moon is tidally locked, but orbital eccentricity and axial tilt cause an apparent wobble called **libration**, revealing ~59% of the lunar surface over time.

- **Libration in Longitude:** Apparent east-west wobble caused by the Moon's elliptical orbit (varying orbital speed vs constant rotation). Related to sub-Earth longitude.
- **Libration in Latitude:** Apparent north-south wobble caused by the $\sim 6.7^\circ$ tilt of the lunar spin axis relative to its orbit. Related to sub-Earth latitude.

WHY UNCALIBRATED IMAGE STACKING FAILS

If Observation A has Sub-Earth (lat = $+5^\circ$, lon = -4°) and Observation B has Sub-Earth (lat = -5° , lon = $+4^\circ$), identical lunar craters appear at different pixel offsets. Simple unaligned image addition produces severe blur and false duplicated structures.

10 POSITION ANGLE VS. LIBRATION

Contributors must strictly distinguish between image plane rotation and 3D viewing tilt:

Rotation in Image (Position Angle)

Orientation issue: "Which way is lunar north pointing on my camera sensor?" Corrected via 2-D in-plane rotation.

Libration

Viewing geometry issue: "Which part of the 3D lunar surface is facing Earth?" Corrected via 3-D spherical realignment.

11 REFERENCE FRAMES VS. COORDINATE SYSTEMS

A **reference frame** defines the physical basis for axes (e.g., a *Moon-fixed frame* attached to and rotating with the Moon). A **coordinate system** defines how points are represented inside that frame (e.g., latitude/longitude vs Cartesian x,y,z).

12 CARTESIAN CONVERSIONS & HAVERSINE DISTANCE

Spherical to Cartesian Code (Python):

```
import numpy as np
lat = np.radians(20)
lon = np.radians(40)
R = 1.0
x = R * np.cos(lat) * np.cos(lon)
y = R * np.cos(lat) * np.sin(lon)
z = R * np.sin(lat)
```

Cartesian to Spherical (Using `atan2`):

Given (x, y, z), calculate distance $r = \sqrt{x^2 + y^2 + z^2}$, `latitude = asin(z/r)`, and `longitude = atan2(y, x)`. **Crucial Rule:** Always use `atan2(y, x)` instead of `atan(y/x)` to preserve quadrant sign info!

Angular Distance on the Moon (Haversine Relation):

The physical separation between two lunar locations (ϕ_1, λ_1) and (ϕ_2, λ_2) is measured along a sphere:

```
a = sin²(Δφ/2) + cos(φ₁) · cos(φ₂) · sin²(Δλ/2)
c = 2 · atan2(√a, √(1-a))
distance = R_lunar · c    (R_lunar ≈ 1737.4 km)
```

13 SUN/EARTH VECTORS & PHASE ANGLE

Normalized direction unit vectors $\hat{S}$ (toward Sun) and $\hat{E}$ (toward Earth) define the phase angle α :

$\cos \alpha = \hat{S} \cdot \hat{E}$ (Sun Direction + Earth Direction → Illumination Geometry → Moon Phase)

14 LOCAL TERRAIN VS. GLOBAL TERMINATOR

Why doesn't a real photograph show an infinitely sharp 1-pixel mathematical terminator line?

- Real terrain contains craters, mountains, valleys, and local slopes that alter local surface normals.
- Atmospheric/telescope dispersion, sensor sampling, noise, and diffuse scattering soften the boundary.

WHY WE USE A GRADIENT BAND

Rather than forcing a brittle 1-pixel edge detector over irregular crater shadows, the pipeline represents the transition as a smooth highlighted gradient band (Dark → Gradient Transition → Bright).

15 FROM LUNAR COORDINATES TO IMAGE PIXELS

Digital images use top-left origin coordinates (u, v) where v increases downward (opposite to standard mathematical x -axes).

Normalized Image Coordinates:

To compare Moon images of different pixel resolutions (e.g., 700px vs 850px), raw pixel coordinates are centered and divided by pixel radius R_{px} :

```
x_n = (x - cx) / Rpx
y_n = (y - cy) / Rpx    # Yields a normalized unit disc centered at (0, 0)
```

Center-Based Rotation Matrix:

To rotate an image by position angle θ without shifting the Moon across the frame, transformation must occur around center (cx, cy) :

```
x_c = x - cx,  y_c = y - cy
x_r = x_c · cos θ - y_c · sin θ
y_r = x_c · sin θ + y_c · cos θ
x' = x_r + cx,  y' = y_r + cy
```

16 OBSERVATION GEOMETRY & METADATA ROLE

The metadata JSON paired with every image is indispensable because the image alone lacks physical scale and orientation metrics:

```
IMAGE + METADATA = PHYSICAL OBSERVATION
```

17 HIERARCHY OF COORDINATE SYSTEMS & SPICE OVERVIEW

LEVEL	COORDINATE CONCEPT	PURPOSE IN PIPELINE
1	Lunar Latitude / Longitude	Locate physical surface features on the Moon.
2	Lunar Cartesian (x, y, z)	Perform 3D vector geometry & dot products.
3	Moon-Fixed Reference Frame	Keep coordinates rigidly attached to rotating Moon body.
4	Observer / View Frame	Describe Earth viewing direction and libration vectors.
5	Image Coordinates (u, v)	Locate raw sensor pixel positions.
6	Normalized Image Coordinates	Compare differently scaled or resized images.

Where SPICE Fits (Advanced Mechanics):

NASA/JPL's SPICE toolkit computes exact spacecraft and planetary geometry. Beginners should master basic spherical vectors first before configuring SPICE kernels.

18 VISIBILITY + ILLUMINATION CLASSIFICATION MATRIX

Comparing surface normal **N** with Earth vector **E** and Sun vector **S** divides the Moon into 4 quadrant regions:

CONDITION	VISIBLE FROM EARTH ($N \cdot E > 0$)	HIDDEN FAR SIDE ($N \cdot E < 0$)
Illuminated ($N \cdot S > 0$)	Visible Daylight (Observed Moon disc)	Hidden Daylight (Farside day)
Unilluminated ($N \cdot S < 0$)	Visible Night (Dark side of disk)	Hidden Night (Farside night)

19 CONNECTING SCIENCE TO PROCESSING STAGES

- Ingest:** Read SVS image + JSON metadata (`time` , `posangle` , `subsolar` , `subearth`).
- Disc Detection:** Estimate center (`cx` , `cy`) and radius `R` .
- Normalization:** Resize diameter & rotate around center by `posangle` .
- Extraction:** Compute brightness gradients to construct smooth terminator band.
- Fusion:** Accumulate complementary multi-geometry observations into master model.

20 SELF-ASSESSMENT Q&A & BEGINNER PITFALLS

Q1: What is the sub-Earth point?

The physical lunar surface location associated with the direction directly toward Earth.

Q2: Why does $\mathbf{N} \cdot \mathbf{S}$ matter?

It measures how directly a surface normal faces the Sun, defining illumination and the terminator boundary ($\mathbf{N} \cdot \mathbf{S} = 0$).

Q3: Why can't we directly stack two Moon photographs?

Their scale, orientation, viewing geometry (libration), and illumination angles differ across dates.

7 Common Beginner Pitfalls to Avoid:

1. **Treating Latitude as Y:** Latitude is an angle on a sphere, not a pixel row height.
2. **Forgetting Radians:** Passing degrees directly into `np.sin()` without `np.radians()`.
3. **Confusing Longitude Direction:** Assuming sign conventions without checking metadata specs.
4. **Inverting Image Y:** Forgetting image pixel rows increase downward, whereas math Y increases upward.
5. **Assuming Perfect Spheres:** Ignoring topographic relief when building high-precision models.
6. **Confusing Sub-Earth Point with Pixel Center:** Treating an astronomical location as a simple pixel offset.
7. **Mishandling Shadow Boundaries:** Interpreting local crater shadows as the global terminator.

21 PRACTICAL LEARNING EXERCISES

Exercise 1: Spherical to Cartesian & Dot Product (Python)

```
import numpy as np

def latlon_to_xyz(lat_deg, lon_deg, radius=1.0):
    lat_r, lon_r = np.radians(lat_deg), np.radians(lon_deg)
    return np.array([
        radius * np.cos(lat_r) * np.cos(lon_r),
        radius * np.cos(lat_r) * np.sin(lon_r),
        radius * np.sin(lat_r)
    ])

N = latlon_to_xyz(20.0, 40.0) # Surface normal
S = latlon_to_xyz(0.0, 30.0) # Sun direction
print(f"N dot S = {np.dot(N, S):.4f}") # >0: Illuminated
```

22 RECOMMENDED STUDY ORDER & RESOURCES

7-Level Progression

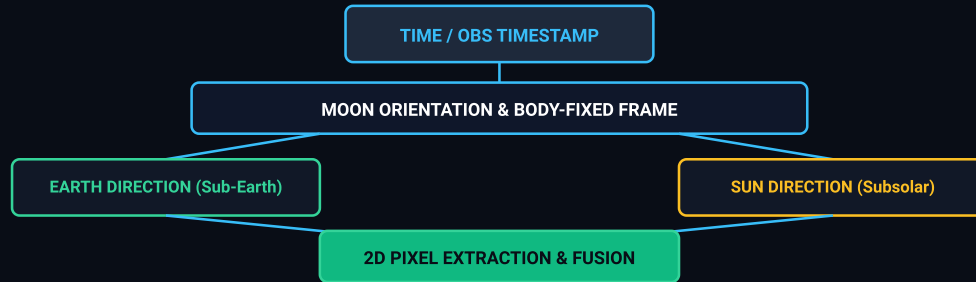
1. Astronomy Basics (Phases, Libration)
2. Lunar Coordinates (Latitude, Longitude)
3. Vector Geometry (Dot products, Normals)
4. Reference Frames (Moon-fixed vs View)
5. Image Geometry (Pixels, Rotation)
6. Pipeline Core (Metadata, Normalization)
7. Advanced Astronomy (SPICE Kernels)

Key Resources

- **NASA SVS Dial-A-Moon:** Visual phase & libration guides.
- **NASA Moon Essentials:** Longitude & Latitude libration breakdowns.
- **NAIF SPICE Tutorials:** Coordinate frames & body-fixed geometry.

23 FINAL MENTAL MODEL & SUMMARY

FIGURE 2: COMPLETE ASTRONOMICAL & COMPUTATIONAL PROCESSING CHAIN



THE CORE QUESTION FOR EVERY CONTRIBUTOR

Whenever writing code for alignment, mask extraction, or fusion, always ask: "What physical point or vector direction in 3D space does this number actually represent?"